T3000 API & Protocol

Message 17: GET_WEBVIEW_LIST · built Nov 13, 2026

Message 17: GET_WEBVIEW_LIST

GET_WEBVIEW_LIST retrieves specific data from BACnet devices. Use this message to read inputs, outputs, variables, programs, PID controllers, or monitors from a device.

Common Uses:

- Loading input values for display
- Reading output states
- Fetching program status
- Getting PID controller setpoints
- Retrieving trendlog configurations

Overview

Message Type: 17 **Action Name:** `GET_WEBVIEW_LIST` **File Location:** `BacnetWebView.cpp` line 1614
Purpose: Selective data retrieval with index range support

Request Format

JSON

```
{
  "action": 17,
  "source": 1,
  "panelId": 0,
  "serialNumber": 12345,
  "entryType": 6,
  "entryIndexStart": 0,
  "entryIndexEnd": 15,
  "objectinstance": 123456
}
```

Parameters

Parameter	Type	Required	Description
`action`	Integer	✅	Must be 17
`source`	Integer	✅	Message source: 0=T3000, 1=WebUI
`panelId`	Integer	✅	Panel number (0-254)
`serialNumber`	Integer	✅	Device serial number
`entryType`	Integer	✅	Type of entry to read (see Entry Types)
`entryIndexStart`	Integer	✅	Starting index (0-based)
`entryIndexEnd`	Integer	✅	Ending index (inclusive)
`objectinstance`	Integer	✅	BACnet object instance

Entry Types

Type	Value	Name	Max Count	Group Size	Command
BAC_IN	1	Inputs	64	8	READINPUT_T3000
BAC_OUT	2	Outputs	64	8	READOUTPUT_T3000
BAC_VAR	3	Variables	64	8	READVARIABLE_T3000
BAC_PRG	6	Programs	16	10	READPROGRAM_T3000
BAC_PID	11	Controllers	16	varies	READCONTROLLER_T3000
BAC_TL	10	Monitors	12	varies	READMONITOR_T3000

Response Format

JSON

```
{
  "action": "GET_WEBVIEW_LIST_RES",
  "data": {
    "panelId": 0,
    "serialNumber": 12345,
    "entryType": 6,
    "entryIndexStart": 0,
    "entryIndexEnd": 15,
    "objectinstance": 123456,
    "device_data": [
      {
        "pid": 0,
        "type": "PROGRAM",
        "index": 0,
        "id": "PRG1",
        "command": "0PRG1",
        "description": "Program 1",
        "label": "PRG1",
        "on_off": 1,
        "auto_manual": 0,
        "com_prg": 0,
        "errcode": 0
      },
      {
        "pid": 0,
        "type": "PROGRAM",
        "index": 1,
        "id": "PRG2",
        "command": "0PRG2",
        "description": "Program 2",
        "label": "PRG2",
        "on_off": 0,
        "auto_manual": 1,
        "com_prg": 0,
        "errcode": 0
      }
      // ... more entries ...
    ]
```

```
  }
}
```

Response Fields by Entry Type

BAC_IN (Inputs)

JSON

```
{
  "pid": 0,
  "type": "INPUT",
  "index": 0,
  "id": "IN1",
  "command": "0IN1",
  "description": "Room Temperature",
  "label": "ROOM_TEMP",
  "value": 720,           // 72.0°F (value * 0.1)
  "auto_manual": 0,       // 0=Auto, 1=Manual
  "filter": 3,
  "control": 1,
  "digital_analog": 0,    // 0=Analog, 1=Digital
  "range": 1,             // Unit type
  "calibration_sign": 0,
  "calibration_h": 0,
  "calibration_l": 0,
  "decom": 0              // 0=Normal, 1=Open, 2=Shorted
}
```

BAC_OUT (Outputs)

JSON

```
{
  "pid": 0,
  "type": "OUTPUT",
  "index": 0,
  "id": "OUT1",
  "command": "0OUT1",
  "description": "Cooling Valve",
  "label": "COOL_VLV",
  "value": 500,           // 50.0%
  "auto_manual": 0,
  "digital_analog": 0,
  "range": 0,
  "control": 1,
  "decom": 0,
  "low_voltage": 0,
  "pwm_period": 0
}
```

BAC_VAR (Variables)

JSON

```
{
  "pid": 0,
  "type": "VARIABLE",
  "index": 0,
  "id": "VAR1",
  "command": "0VAR1",
  "description": "Setpoint",
  "label": "SP",
  "value": 720,
  "auto_manual": 0,
  "digital_analog": 0,
  "range": 1,
  "control": 0,
  "unused": 0
}
```

BAC_PRG (Programs)

JSON

```
{
  "pid": 0,
  "type": "PROGRAM",
  "index": 0,
  "id": "PRG1",
  "command": "0PRG1",
  "description": "Morning Startup",
  "label": "MORN_UP",
  "on_off": 1,            // 0=Off, 1=On
  "auto_manual": 0,       // 0=Auto, 1=Manual
  "com_prg": 0,
  "errcode": 0
}
```

BAC_PID (Controllers)

JSON

```
{
  "pid": 0,
  "type": "CONTROLLER",
  "index": 0,
  "id": "PID1",
  "command": "0PID1",
  "input": 0,             // Input point index
  "input_value": 720,
  "value": 500,           // Output value
  "setpoint": 0,          // Setpoint point index
  "setpoint_value": 720,
  "units": 1,
  "auto_manual": 0,
  "action": 0,            // 0=Direct, 1=Reverse
  "proportional": 50,     // P gain
  "reset": 0,             // I gain
  "bias": 0,
  "rate": 0               // D gain
}
```

BAC_TL (Monitors/Trendlogs)

JSON


```
{
  "pid": 0,
  "type": "MONITOR",
  "index": 0,
  "id": "TL1",
  "command": "0TL1",
  "label": "Temp Log",
  "num_inputs": 2,
  "an_inputs": 2,
  "status": 0,
  "second_interval_time": 0,
  "minute_interval_time": 5,
  "hour_interval_time": 0,
  "max_time_length": 1440,
  "next_sample_time": 0,
  "inputs": [0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
  "range": [1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
}
```


Implementation Details

Code Location

File: `T3000-Source/T3000/BacnetWebView.cpp` **Line:** 1614-1955 **Function:** `HandleWebViewMsg()`

Processing Flow

TEXT

```
1. Parse JSON request
   ↓
2. Validate parameters (panelId, objectinstance, indices)
   ↓
3. Check cache for panel data
   ↓
4. Switch on entryType
   ↓
5. Calculate read groups (chunks)
   ↓
6. Loop through groups:
   a. Calculate chunk start/end
   b. Call GetPrivateDataSaveSPBlocking()
   c. Copy data from temp buffer to global array
   d. Build JSON response for this chunk
   ↓
7. Return complete JSON response
```

Reading Algorithm

C++

```
case BAC_PRG:  // Programs
{
    if (entry_index_end >= entry_index_start)
    {
        // Step 1: Calculate groups
        int totalCount = entry_index_end - entry_index_start + 1;
        int groupSize = BAC_READ_PROGRAM_GROUP_NUMBER;  // 10
        int read_program_group = (totalCount + groupSize - 1) / groupSize;

        // Step 2: Loop through groups
        for (int i = 0; i < read_program_group; ++i)
        {
            int temp_start = entry_index_start + i * groupSize;
            if (temp_start > entry_index_end)
                break;

            int temp_end = temp_start + groupSize - 1;
            if (temp_end > entry_index_end)
                temp_end = entry_index_end;

            // Boundary check
            if (temp_start >= BAC_PROGRAM_ITEM_COUNT)
                break;
            if (temp_end >= BAC_PROGRAM_ITEM_COUNT)
                temp_end = BAC_PROGRAM_ITEM_COUNT - 1;

            // Step 3: Blocking read from device
            if (GetPrivateDataSaveSPBlocking(entry_objectinstance,
                READPROGRAM_T3000,
                (uint8_t)temp_start,
                (uint8_t)temp_end,
                sizeof(Str_program_point),
                4) > 0)  // 4 retries
            {
                // Step 4: Copy to global array
                for (int idx = temp_start; idx <= temp_end; ++idx)
                {
                    if (idx < BAC_PROGRAM_ITEM_COUNT) {
```

```
                        g_Program_data[temp_panel_id].at(idx) = s_Program_data[idx];
                    }
                }

                Sleep(SEND_COMMAND_DELAY_TIME);  // 200ms delay
            }
            else
            {
                // Handle read failure
                continue;
            }

            // Step 5: Build JSON for this chunk
            int npanel_id = temp_panel_id;
            for (int idx = temp_start; idx < temp_end; idx++)
            {
                tempjson["data"]["device_data"][point_idx]["pid"] = npanel_id;
                tempjson["data"]["device_data"][point_idx]["type"] = "PROGRAM";
                // ... populate all fields ...
                point_idx++;
            }
        }
    }
    break;
}
```

Group Sizes (Chunking)

Reading data in chunks prevents network overload:

Entry Type	Group Size	Example: Reading 0-15
Inputs	8	2 groups: [0-7], [8-15]
Outputs	8	2 groups: [0-7], [8-15]
Variables	8	2 groups: [0-7], [8-15]
Programs	10	2 groups: [0-9], [10-15]
Controllers	varies	Depends on configuration
Monitors	varies	Depends on configuration

Data Flow

```
TEXT
```

```
Device → BACnet Protocol → GetPrivateDataSaveSPBlocking()
                                    ↓
                        s_Program_data[] (temp buffer)
                                    ↓
                        g_Program_data[panelId][] (global cache)
                                    ↓
                        JSON Response → Rust API → Frontend
```

Error Handling

Common Errors

1. Device Offline

```
JSON
```

```
{
  "action": "ERROR_RES",
  "error": {
    "code": -1,
    "message": "Read timeout",
    "details": "ERROR: Read program failed start=0, end=9"
  }
}
```

2. Invalid Index Range

```
JSON
```

```
{
  "action": "ERROR_RES",
  "error": {
    "code": -2,
    "message": "Invalid parameter",
    "details": "entryIndexEnd (20) exceeds BAC_PROGRAM_ITEM_COUNT (16)"
  }
}
```

3. Object Instance Mismatch

JSON

```
{
  "action": "ERROR_RES",
  "error": {
    "code": -5,
    "message": "Object mismatch",
    "details": "Expected objectinstance=123456, got 123457"
  }
}
```

Performance

Timing

Scenario	Typical Time
Cache hit (no device read)	< 50ms
Fresh read (1-10 items)	200-500ms
Fresh read (full range 0-15)	1-2 seconds
Multiple groups (chunked)	+200ms per group

Optimization Tips

1. **Request only needed range** - Don't read 0-63 if you only need 0-15
2. **Batch requests** - Read multiple items at once vs individual requests
3. **Use cache when possible** - Check if fresh data is needed
4. **Avoid overlapping requests** - Wait for response before new request

Usage Examples

Example 1: Read All Programs

JSON

```
{
  "action": 17,
  "source": 1,
  "panelId": 0,
  "serialNumber": 12345,
  "entryType": 6,
  "entryIndexStart": 0,
  "entryIndexEnd": 15,
  "objectinstance": 123456
}
```

Example 2: Read Single Input

JSON

```
{
  "action": 17,
  "source": 1,
  "panelId": 0,
  "serialNumber": 12345,
  "entryType": 1,
  "entryIndexStart": 0,
  "entryIndexEnd": 0,
  "objectinstance": 123456
}
```

Example 3: Read Controllers 0-7

```JSON
{
  "action": 17,
  "source": 1,
  "panelId": 0,
  "serialNumber": 12345,
  "entryType": 11,
  "entryIndexStart": 0,
  "entryIndexEnd": 7,
  "objectinstance": 123456
}
```

Comparison with Other Messages

vs GET_PANEL_DATA

- **GET_PANEL_DATA:** Loads ALL cached data for entire panel
- **GET_WEBVIEW_LIST:** Reads specific entry type with range

Use GET_WEBVIEW_LIST when: You need specific data, especially fresh from device **Use GET_PANEL_DATA when:** Initial panel load, need all data types

vs GET_ENTRIES

- **GET_ENTRIES:** Read specific individual entries by exact indices
- **GET_WEBVIEW_LIST:** Read continuous range of one entry type

Use GET_WEBVIEW_LIST when: Reading sequential range **Use GET_ENTRIES when:** Reading scattered individual entries

See Also

- Message Index →
- UPDATE_WEBVIEW_LIST →
- GET_PANEL_DATA →
- BACnet Commands →
- Data Structures →